

Expt. No. : 9

Date :

INDEXING AND SORTING

AIM

To write a python program to implement indexing and sorting using numpy packages.

SOFTWARE REQUIRED

Python 3.14

THEORY

NumPy provides powerful tools for indexing and sorting arrays. Indexing allows for accessing and modifying elements within an array, while sorting organizes the elements in a specified order. Key concepts include:

INDEXING

- Basic Indexing: Access elements using indices.
- Slicing: Retrieve a range of elements from an array.
- Boolean Indexing: Filter elements based on conditions.
- Fancy Indexing: Access multiple elements using an array of indices.

SORTING

- Sorting Arrays: Organizes the elements in ascending or descending order.
- In-place Sorting: Sorts the original array without creating a new one.
- Sorting along an Axis: Sorts elements along rows or columns in multi-dimensional arrays.
- Stable Sort: Maintains the original order of equal elements.

PROBLEM

1. Build a Python program to accept a list of numbers and use indexing to extract the even-indexed and odd-indexed elements, then sort each resulting list.
2. An electric vehicle battery management system records the voltage of individual cells in a battery pack. Build a Python program to access specific cell voltages using indexing and sort the readings to identify cells with the highest and lowest voltage.
3. Logan is cleaning his apartment. In particular, he must sort his old favorite sequence, P , of N positive integers in nondecreasing order. He's tired from a long day, so he invented an easy way (in his opinion) to do this job. His algorithm can be described by the following pseudocode:

```
while isNotSorted(P) do {
    WaitOneMinute();
    RandomShuffle(P)
}
```

Can you determine the expected number of minutes that Logan will spend waiting for P to be sorted?

ALGORITHM

- STEP 1:** Start the program.
- STEP 2:** Import the NumPy package as np.
- STEP 3:** Get the values for rows and columns from the user.
- STEP 4:** Get the array elements row-wise from the user.
- STEP 5:** Convert the list of elements to a NumPy array.
- STEP 6:** Display the original array.
- STEP 7:** Demonstrate basic indexing by accessing a specific element.
- STEP 8:** Demonstrate slicing by retrieving a range of elements.
- STEP 9:** Demonstrate boolean indexing by filtering elements based on a condition.
- STEP 10:** Demonstrate fancy indexing by accessing multiple specified elements.
- STEP 11:** Sort the array and display the sorted result.
- STEP 12:** Sort the array along an axis and display the sorted result.

PROGRAM CODING

```

numbers = list(map(int, input("Enter numbers separated by space:
").split()))
even_indexed = sorted(numbers[0::2])
odd_indexed = sorted(numbers[1::2])
print("Sorted Even-indexed elements:", even_indexed)
print("Sorted Odd-indexed elements:", odd_indexed)

```

OUTCOMES OF PROGRAM CODING

```

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on
win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:\Users\Mugil\Desktop\Python\Experiment\Experiment9.py =====
Enter numbers separated by space: 8 3 56 78 49 51 57
Sorted Even-indexed elements: [8, 49, 56, 57]
Sorted Odd-indexed elements: [3, 51, 78]
>>>

```

PROBLEM CODING

```

import numpy as np
voltages = np.array(list(map(float, input("Enter cell voltages separated by
space: ").split()))))
print(f"First Cell Voltage: {voltages[0]} V")
print(f>Last Cell Voltage: {voltages[-1]} V")
sorted_v = np.sort(voltages)
print("Sorted Voltages:", sorted_v)
print(f"Lowest Cell Voltage: {sorted_v[0]} V")
print(f"Highest Cell Voltage: {sorted_v[-1]} V")

```

OUTCOMES OF PROGRAM CODING

```

Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)]
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Problem\experiment9.py =====
Enter cell voltages separated by space: 3.7 3.5 3.6 3.2 3.9
First Cell Voltage: 3.7 V
Last Cell Voltage: 3.9 V
Sorted Voltages: [3.2 3.5 3.6 3.7 3.9]
Lowest Cell Voltage: 3.2 V
Highest Cell Voltage: 3.9 V
>>>

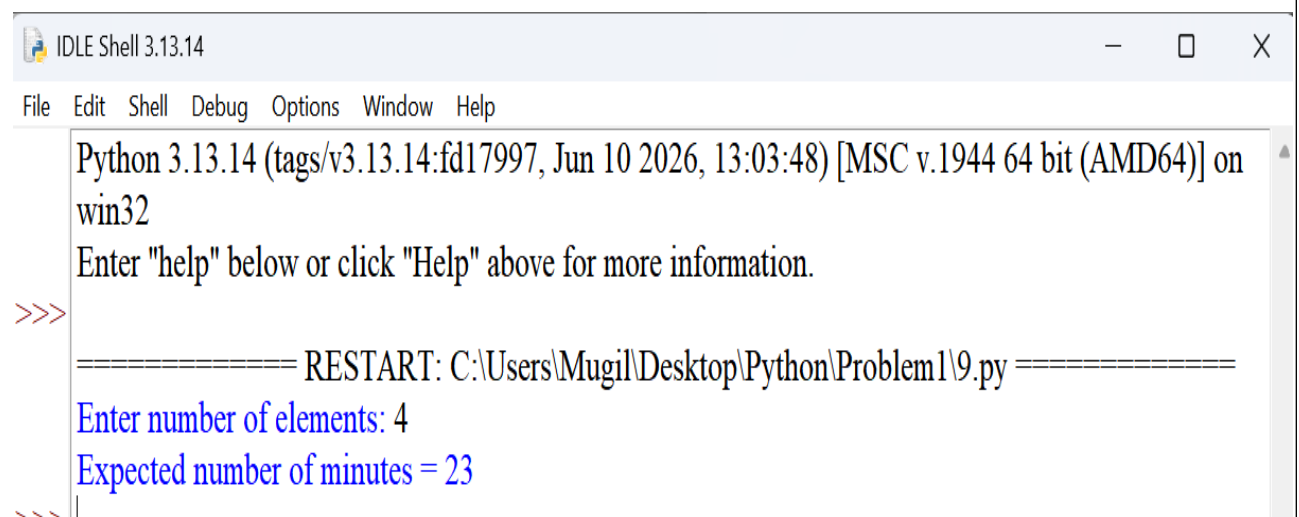
```

PROBLEM CODING

```
import math
```

```
n = int(input("Enter number of elements: "))  
answer = math.factorial(n) - 1  
print("Expected number of minutes =", answer)
```

OUTCOMES OF PROGRAM CODING



```
IDLE Shell 3.13.14  
File Edit Shell Debug Options Window Help  
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on  
win32  
Enter "help" below or click "Help" above for more information.  
>>>  
===== RESTART: C:\Users\Mugil\Desktop\Python\Problem1\9.py =====  
Enter number of elements: 4  
Expected number of minutes = 23  
>>>
```

VIVA QUESTION

1. What is indexing in Python, and how is it used with lists and arrays?

Indexing in Python refers to the method of accessing individual elements or slices of a collection, such as lists or arrays. Each element in a list or array has a unique index, starting from 0 for the first element. For example, to access the third element in a list `my_list`, you would use `my_list[2]`. In the case of NumPy arrays, indexing works similarly, allowing access to specific elements or sub-arrays.

2. Question: How can you perform slicing on a list in Python? Provide an example.

Slicing allows you to retrieve a subset of a list by specifying a start and end index. The syntax is `list[start:end]`, where `start` is inclusive and `end` is exclusive. For example, given the list `my_list = [1, 2, 3, 4, 5]`, the slice `my_list[1:4]` would return `[2, 3, 4]`.

3. Question: Explain the difference between sorting a list using the `sort()` method and the `sorted()` function in Python.

The `sort()` method sorts a list in place and modifies the original list, returning `None`. For example, `my_list.sort()` sorts `my_list` directly. In contrast, the `sorted()` function returns a new sorted list from the original iterable, leaving the original list unchanged. For instance, `sorted_list = sorted(my_list)` creates a new sorted list while `my_list` remains the same.

4. Question: How can you sort a NumPy array, and what is the difference between sorting along different axes?

You can sort a NumPy array using the `np.sort()` function or the `sort()` method of a NumPy array. The key difference when sorting along different axes is that specifying `axis=0` sorts along columns, while `axis=1` sorts along rows. For example, `np.sort(array, axis=0)` sorts each column independently, whereas `np.sort(array, axis=1)` sorts each row independently.

5. Question: What is the role of the `key` parameter in the `sorted()` function, and how can it be used?

The `key` parameter in the `sorted()` function allows you to specify a function that returns a value used for sorting. This is particularly useful for sorting complex data types like lists of tuples or dictionaries. For instance, if you have a list of tuples representing `(name, age)`, you can sort by age by using `sorted(my_list, key=lambda x: x[1])`, where `x[1]` refers to the age element in each tuple.

RESULT:

Thus, the Python program successfully implemented indexing and sorting operations on the given list of numerical values. The required elements

were accessed using positive and negative indexing, and the values were correctly arranged in ascending and descending order. The program was executed successfully, and the output was verified.

INFERENCE:

The experiment demonstrates that Python indexing enables efficient access to specific elements in a list, while sorting helps organize numerical data systematically. The successful execution validates their usefulness in analysing and arranging Electrical Engineering measurements for effective data interpretation.

Assessment	Max. Marks	Marks Obtained
Conduction of Practical	15	
Report	5	
Viva	5	
Total Marks	25	
Signature of Lab In-Charge		